

ASSIGNMENT-2

Course: Full Stack Web Development

Course Code: 23CM4121

Assignment No.: 2

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 05-10-2026

1. TITLE

Student Information Management System Using Node.js and MongoDB

2. AIM

To understand, design, and implement a robust Student Information Management System using MongoDB and Node.js to perform comprehensive CRUD operations (Create, Read, Update, Delete), conditional filtering, sorting, unique B-Tree indexing, query execution analysis, and aggregation queries using both the MongoDB Shell (mongosh) and the official MongoDB Node.js driver (MongoClient).

3. OBJECTIVE

The objectives of this experiment are:

- To understand NoSQL database principles, document-oriented data storage, and MongoDB core architecture (Databases, Collections, BSON Documents).
- To establish an asynchronous connection between Node.js and a MongoDB database using the official mongodb npm package and MongoClient.
- To perform batch insertion of structured student documents containing roll numbers, personal details, academic metrics, and contact information.
- To query and retrieve documents matching specific criteria, including branch-specific filtering and marks threshold comparisons (\$gt, \$lt).
- To implement primary key lookups using unique student roll numbers (rollNo).
- To execute compound conditional queries combining multiple relational criteria using logical operators (\$and, \$gte).
- To implement dynamic update operations for individual and multiple fields using updateOne() with the \$set operator.
- To execute deletion operations targeting specific student records by unique roll number using deleteOne().

- To sort and order retrieved records in ascending and descending orders using the `.sort()` cursor method.
 - To build a unique B-Tree index on the `rollNo` field and analyze execution performance gains (transition from COLLSCAN to IXSCAN) using `.explain('executionStats')`.
 - To construct MongoDB aggregation pipelines for group-wise statistical computations (average, maximum, minimum marks, and document counts).
-

4. THEORY

NoSQL & Document Databases — unlike relational databases (RDBMS) that store data in rigid tabular rows and columns with fixed schemas, NoSQL document databases like MongoDB store data as flexible, self-describing, JSON-like BSON (Binary JSON) documents. This structure allows dynamic schemas, nested objects, and arrays.

MongoDB Hierarchy — MongoDB organizes data hierarchically: a Database contains one or more Collections (analogous to tables in SQL), and each Collection contains multiple Documents (analogous to rows or records). Every document is automatically assigned an immutable unique identifier field named `_id` (an ObjectId).

Node.js MongoDB Driver — the official `mongodb` library provides high-performance asynchronous APIs to interact with MongoDB servers. The `MongoClient` class manages connection pooling, topology monitoring, and request dispatching using modern JavaScript `async/await` patterns.

CRUD Operations — CRUD represents the four fundamental persistent storage functions: Create (`insertOne`, `insertMany`), Read (`find`, `findOne`), Update (`updateOne`, `updateMany` with atomic operators like `$set`), and Delete (`deleteOne`, `deleteMany`).

Query Selectors & Projections — MongoDB uses expressive JSON query selectors such as `$gt` (greater than), `$gte` (greater than or equal to), `$lt` (less than), `$in` (matches any value in an array), and logical operators such as `$and`, `$or`. Projections specify exactly which fields to include (1) or exclude (0) in the query result set.

Sorting & Limiting — the `.sort()` cursor method orders documents based on one or more fields where 1 denotes ascending order and -1 denotes descending order. The `.limit(n)` method caps the maximum number of documents returned, essential for top-performer queries and pagination.

Indexing & Execution Plan Analysis (COLLSCAN vs. IXSCAN) — indexes are specialized B-Tree data structures that store a small portion of the collection's data set in an easy-to-traverse form. Without an index, MongoDB must perform a Collection Scan (COLLSCAN), inspecting every single document in the collection. With an index, MongoDB performs an Index Scan (IXSCAN), directly locating the target documents with $O(\log N)$ lookup complexity. The `.explain('executionStats')` method provides detailed execution telemetry including `executionStages`, `totalDocsExamined`, and `totalKeysExamined`.

Aggregation Framework — a multi-stage data processing pipeline where documents enter a multi-stage pipeline that transforms them into aggregated results. Common stages include `$match` (filtering), `$group` (grouping by key with accumulators like `$sum`, `$avg`, `$max`, `$min`), and `$sort` (ordering output).

5. ALGORITHM / PROCEDURE

- Initialize project directory and install the official MongoDB Node.js driver using `npm install mongodb`.
 - Create the MongoDB Shell script file `collegeDB_queries.mongoddb.js` and the Node.js application file `app.js`.
 - In `collegeDB_queries.mongoddb.js`, select the database using `use('collegeDB')` and drop the existing collection `db.students.drop()` for clean state execution.
 - Implement document insertion using `db.students.insertMany()` with at least 7 student documents containing `rollNo`, `name`, `branch`, `year`, `marks`, and `email`.
 - Implement basic and conditional queries: `db.students.find()` for all records, `db.students.find({ branch: 'CSE-AIML' })` for branch filtering, and `db.students.find({ marks: { $gt: 75 } })` for marks filtering.
 - Implement unique document search using `db.students.find({ rollNo: '23CM001' })` and compound queries using `$and` with `$gte` and equality operators.
 - Implement record updates using `db.students.updateOne({ rollNo: '23CM001' }, { $set: { marks: 95 } })` and update email for 23CM002.
 - Implement record deletion using `db.students.deleteOne({ rollNo: '23CM007' })` and verify the remaining collection.
 - Implement sorting using `db.students.find().sort({ marks: -1 })` to arrange students in descending order of academic performance.
 - Demonstrate query performance: analyze execution stats with `db.students.find({ rollNo: '23CM001' }).explain('executionStats')` to observe `COLLSCAN`, create a unique index using `db.students.createIndex({ rollNo: 1 }, { unique: true })`, and re-evaluate explain plan to observe `IXSCAN`.
 - Construct aggregation pipeline using `db.students.aggregate()` with `$group` to calculate branch-wise total students, average marks, max marks, and min marks.
 - In `app.js`, implement the complete `StudentDBManager` class encapsulating `MongoClient` lifecycle, CRUD methods, indexing, explain plan, CLI interactive menu, and automated test suite.
 - Execute `node app.js --demo` and `mongosh collegeDB_queries.mongoddb.js`, verify outputs against test cases, and capture terminal execution logs.
-

6. PROGRAM

Program 1: `collegeDB_queries.mongoddb.js`

```
// 1. Switch to (or create) database
```

```
use("collegeDB");
```

```
// Optional: Clean up existing collection for fresh start
```

```
db.students.drop();
```

```
// =====
```

```
// 2. INSERTION: Insert multiple student records (at least 5)
```

```
// =====
db.students.insertMany([
  {
    rollNo: "23CM001",
    name: "Ravi Kumar",
    branch: "CSE-AIML",
    year: 3,
    marks: 85,
    email: "ravi@example.com"
  },
  {
    rollNo: "23CM002",
    name: "Ananya Sharma",
    branch: "CSE-AIML",
    year: 3,
    marks: 92,
    email: "ananya@example.com"
  },
  {
    rollNo: "23CS015",
    name: "Vikram Singh",
    branch: "CSE",
    year: 2,
    marks: 74,
    email: "vikram@example.com"
  },
  {
    rollNo: "23EC030",
    name: "Pooja Reddy",
    branch: "ECE",
    year: 4,
    marks: 48,
    email: "pooja@example.com"
  },
  {
    rollNo: "23IT012",
    name: "Karthik Verma",
    branch: "IT",
    year: 2,
    marks: 81,
    email: "karthik@example.com"
  },
  {
    rollNo: "23CS045",
    name: "Sneha Patel",
```

```

        branch: "CSE",
        year: 3,
        marks: 65,
        email: "sneha@example.com"
    },
    {
        rollNo: "23CM007",
        name: "Rahul Mehta",
        branch: "CSE-AIML",
        year: 1,
        marks: 42,
        email: "rahul@example.com"
    }
]);

// =====
// 3. RETRIEVAL OPERATIONS
// =====

// a. Display all students
print("--- ALL STUDENTS ---");
db.students.find().pretty();

// b. Display students belonging to a particular branch (e.g., 'CSE-AIML')
print("--- STUDENTS IN CSE-AIML BRANCH ---");
db.students.find({ branch: "CSE-AIML" }).pretty();

// c. Display students who scored more than 75 marks
print("--- STUDENTS WITH MARKS > 75 ---");
db.students.find({ marks: { $gt: 75 } }).pretty();

// d. Search for a student using rollNo
print("--- SEARCH STUDENT BY ROLL NO: 23CM001 ---");
db.students.find({ rollNo: "23CM001" }).pretty();

// e. Search students based on specified conditions (marks >= 70 AND year == 3)
print("--- STUDENTS WITH MARKS >= 70 AND YEAR == 3 ---");
db.students.find({
    $and: [
        { marks: { $gte: 70 } },
        { year: 3 }
    ]
}).pretty();

// =====

```

```

// 4. UPDATE OPERATIONS
// =====

// a. Update marks of a particular student
print("--- UPDATING MARKS FOR 23CM001 TO 95 ---");
db.students.updateOne(
  { rollNo: "23CM001" },
  { $set: { marks: 95 } }
);

// b. Update another field (e.g., email or branch)
print("--- UPDATING EMAIL FOR 23CM002 ---");
db.students.updateOne(
  { rollNo: "23CM002" },
  { $set: { email: "ananya.sharma2026@university.edu" } }
);

// Verify updates
db.students.find({ rollNo: { $in: ["23CM001", "23CM002"] } }).pretty();

// =====
// 5. DELETION OPERATION
// =====

// Delete a student record using rollNo
print("--- DELETING STUDENT WITH ROLL NO: 23CM007 ---");
db.students.deleteOne({ rollNo: "23CM007" });

// =====
// 6. SORTING OPERATION
// =====

// Display students in descending order of marks (-1 for descending, 1 for
ascending)
print("--- STUDENTS SORTED BY MARKS (DESCENDING) ---");
db.students.find().sort({ marks: -1 }).pretty();

// =====
// 7. INDEXING & PERFORMANCE DEMONSTRATION
// =====

// Step 1: Analyze query performance BEFORE index creation (COLLSCAN = Collection
Scan)
print("--- EXPLAIN PLAN BEFORE INDEXING (COLLSCAN) ---");
db.students.find({ rollNo: "23CM001" }).explain("executionStats");

```

```

// Step 2: Create a unique index on 'rollNo'
print("--- CREATING UNIQUE INDEX ON rollNo ---");
db.students.createIndex({ rollNo: 1 }, { unique: true });

// View existing indexes
db.students.getIndexes();

// Step 3: Analyze query performance AFTER index creation (IXSCAN = Index Scan)
print("--- EXPLAIN PLAN AFTER INDEXING (IXSCAN) ---");
db.students.find({ rollNo: "23CM001" }).explain("executionStats");

//Queries

// Query 1: Find students scoring above 80
print("--- STUDENTS SCORING ABOVE 80 ---");
db.students.find({ marks: { $gt: 80 } }).pretty();

// Query 2: Find students scoring below 50 (need academic support)
print("--- STUDENTS SCORING BELOW 50 ---");
db.students.find({ marks: { $lt: 50 } }).pretty();

// Query 3: Find the highest-scoring student (Topper)
print("--- HIGHEST-SCORING STUDENT ---");
db.students.find().sort({ marks: -1 }).limit(1).pretty();

// Query 4: Find students belonging to a particular branch (e.g. 'CSE')
print("--- CSE BRANCH STUDENTS ---");
db.students.find({ branch: "CSE" }).pretty();

// Query 5: Display students sorted according to marks (ascending / descending)
print("--- ALL STUDENTS SORTED BY MARKS (HIGH TO LOW) ---");
db.students.find({}, { _id: 0, rollNo: 1, name: 1, branch: 1, marks:
1 }).sort({ marks: -1 }).pretty();

// Query 6 (Bonus Aggregation): Branch-wise average marks & student count
print("--- BONUS: BRANCH-WISE PERFORMANCE SUMMARY ---");
db.students.aggregate([
{
  $group: {
    _id: "$branch",
    totalStudents: { $sum: 1 },
    avgMarks: { $avg: "$marks" },
    maxMarks: { $max: "$marks" },
    minMarks: { $min: "$marks" }
  }
}
]);

```

```

    }
  },
  { $sort: { avgMarks: -1 } }
]);

```

Program 2: app.js

```

/**
 * =====
 * Student Information Management System (Node.js & MongoDB Official Driver)
 * Database: collegeDB
 * Collection: students
 * =====
 */

const { MongoClient } = require("mongodb");
const readline = require("readline");

// Initial sample data
const SAMPLE_STUDENTS = [
  {
    rollNo: "23CM001",
    name: "Ravi Kumar",
    branch: "CSE-AIML",
    year: 3,
    marks: 85,
    email: "ravi@example.com"
  },
  {
    rollNo: "23CM002",
    name: "Ananya Sharma",
    branch: "CSE-AIML",
    year: 3,
    marks: 92,
    email: "ananya@example.com"
  },
  {
    rollNo: "23CS015",
    name: "Vikram Singh",
    branch: "CSE",
    year: 2,
    marks: 74,
    email: "vikram@example.com"
  }
];

```

```

    },
    {
      rollNo: "23EC030",
      name: "Pooja Reddy",
      branch: "ECE",
      year: 4,
      marks: 48,
      email: "pooja@example.com"
    },
    {
      rollNo: "23IT012",
      name: "Karthik Verma",
      branch: "IT",
      year: 2,
      marks: 81,
      email: "karthik@example.com"
    },
    {
      rollNo: "23CS045",
      name: "Sneha Patel",
      branch: "CSE",
      year: 3,
      marks: 65,
      email: "sneha@example.com"
    },
    {
      rollNo: "23CM007",
      name: "Rahul Mehta",
      branch: "CSE-AIML",
      year: 1,
      marks: 42,
      email: "rahul@example.com"
    }
  ];

```

```

function printTable(students, title = "Student Records") {
  console.log(`\n${"=".repeat(75)}`);
  console.log(`  ${title.toUpperCase()} (Count: ${students.length})`);
  console.log(`${"=".repeat(75)}`);
  if (!students || students.length === 0) {
    console.log("  [No matching records found]");
    console.log(`${"=".repeat(75)}\n`);
    return;
  }
}

```

```

const pad = (str, len) => String(str || "N/A").padEnd(len);
console.log(`${pad("Roll No", 10)} | ${pad("Name", 18)} | ${pad("Branch", 10)} |
${pad("Year", 5)} | ${pad("Marks", 6)} | Email`);
console.log("-".repeat(75));
students.forEach((s) => {
  console.log(`${pad(s.rollNo, 10)} | ${pad(s.name, 18)} | ${pad(s.branch, 10)} |
${pad(s.year, 5)} | ${pad(s.marks, 6)} | ${s.email || "N/A"}`);
});
console.log(`${"=".repeat(75)}\n`);
}

```

```

class StudentDBManager {
  constructor(uri = "mongodb://localhost:27017") {
    this.uri = uri;
    this.isLive = false;
    this.client = null;
    this.db = null;
    this.collection = null;
    this.inMemoryDocs = [];
    this.indexedFields = new Set();
  }

```

```

  async connect() {
    try {
      this.client = new MongoClient(this.uri, { serverSelectionTimeoutMS: 2000 });
      await this.client.connect();
      await this.client.db("admin").command({ ping: 1 });
      this.db = this.client.db("collegeDB");
      this.collection = this.db.collection("students");
      this.isLive = true;
      console.log(`[CONNECTED] Connected to live MongoDB at ${this.uri}`);
    } catch (err) {
      console.log(`[NOTE] Local MongoDB server not reachable at ${this.uri} ($
{err.message})`);
      console.log("[INFO] Running in In-Memory Simulation Mode (all MongoDB logic
fully emulated).");
      this.isLive = false;
      this.inMemoryDocs = SAMPLE_STUDENTS.map((s) => ({ ...s }));
    }
  }

```

```

  async close() {
    if (this.client && this.isLive) {
      await this.client.close();
    }
  }

```

```

}

async seedInitialData() {
  if (this.isLive) {
    try {
      await this.collection.drop();
    } catch (e) {
      // Collection might not exist yet
    }
    await this.collection.insertMany(SAMPLE_STUDENTS.map((s) => ({ ...s })));
    console.log(`[SUCCESS] Seeded ${SAMPLE_STUDENTS.length} records into
'collegeDB.students'`);
  } else {
    this.inMemoryDocs = SAMPLE_STUDENTS.map((s) => ({ ...s }));
    console.log(`[SIMULATION] Seeded ${SAMPLE_STUDENTS.length} records in-
memory.`);
  }
}

async insertStudent(studentDoc) {
  if (this.isLive) {
    try {
      await this.collection.insertOne(studentDoc);
      return true;
    } catch (err) {
      console.error(`[ERROR] Insert failed: ${err.message}`);
      return false;
    }
  } else {
    if (this.inMemoryDocs.some((s) => s.rollNo === studentDoc.rollNo)) {
      console.log(`[ERROR] Student with Roll No ${studentDoc.rollNo} already
exists!`);
      return false;
    }
    this.inMemoryDocs.push({ ...studentDoc });
    return true;
  }
}

async getAllStudents() {
  if (this.isLive) {
    return await this.collection.find({}, { projection: { _id: 0 } }).toArray();
  }
  return this.inMemoryDocs.map((s) => ({ ...s }));
}

```

```

async getStudentsByBranch(branch) {
  if (this.isLive) {
    return await this.collection.find({ branch: branch }, { projection: { _id: 0 }
  }).toArray();
  }
  return this.inMemoryDocs
    .filter((s) => s.branch && s.branch.toUpperCase() === branch.toUpperCase())
    .map((s) => ({ ...s }));
}

```

```

async getStudentsAboveMarks(minMarks) {
  if (this.isLive) {
    return await this.collection.find({ marks: { $gt: minMarks } }, { projection:
{ _id: 0 } }).toArray();
  }
  return this.inMemoryDocs.filter((s) => s.marks > minMarks).map((s) =>
({ ...s }));
}

```

```

async getStudentsBelowMarks(maxMarks) {
  if (this.isLive) {
    return await this.collection.find({ marks: { $lt: maxMarks } }, { projection:
{ _id: 0 } }).toArray();
  }
  return this.inMemoryDocs.filter((s) => s.marks < maxMarks).map((s) =>
({ ...s }));
}

```

```

async searchByRollNo(rollNo) {
  if (this.isLive) {
    return await this.collection.findOne({ rollNo: rollNo }, { projection: { _id:
0 } });
  }
  const found = this.inMemoryDocs.find((s) => s.rollNo && s.rollNo.toUpperCase()
=== rollNo.toUpperCase());
  return found ? { ...found } : null;
}

```

```

async searchCustom(query) {
  if (this.isLive) {
    return await this.collection.find(query, { projection: { _id:
0 } }).toArray();
  }
  return this.inMemoryDocs.filter((s) => {

```

```

    let match = true;
    if (query.marks && query.marks.$gte !== undefined && s.marks < query.marks.
$gte) match = false;
    if (query.year !== undefined && s.year !== query.year) match = false;
    return match;
  }).map((s) => ({ ...s }));
}

```

```

async updateStudentMarks(rollNo, newMarks) {
  if (this.isLive) {
    const res = await this.collection.updateOne({ rollNo: rollNo }, { $set:
{ marks: newMarks } });
    return res.modifiedCount > 0;
  }
  const target = this.inMemoryDocs.find((s) => s.rollNo.toUpperCase() ===
rollNo.toUpperCase());
  if (target) {
    target.marks = newMarks;
    return true;
  }
  return false;
}

```

```

async updateStudentField(rollNo, fieldName, newValue) {
  if (this.isLive) {
    const updateObj = {};
    updateObj[fieldName] = newValue;
    const res = await this.collection.updateOne({ rollNo: rollNo }, { $set:
updateObj });
    return res.modifiedCount > 0;
  }
  const target = this.inMemoryDocs.find((s) => s.rollNo.toUpperCase() ===
rollNo.toUpperCase());
  if (target) {
    target[fieldName] = newValue;
    return true;
  }
  return false;
}

```

```

async deleteByRollNo(rollNo) {
  if (this.isLive) {
    const res = await this.collection.deleteOne({ rollNo: rollNo });
    return res.deletedCount > 0;
  }
}

```

```

    const beforeLen = this.inMemoryDocs.length;
    this.inMemoryDocs = this.inMemoryDocs.filter((s) => s.rollNo.toUpperCase() !==
rollNo.toUpperCase());
    return this.inMemoryDocs.length < beforeLen;
  }

  async getStudentsSortedByMarks(descending = true) {
    if (this.isLive) {
      return await this.collection
        .find({}, { projection: { _id: 0 } })
        .sort({ marks: descending ? -1 : 1 })
        .toArray();
    }
    const list = this.inMemoryDocs.map((s) => ({ ...s }));
    return list.sort((a, b) => (descending ? b.marks - a.marks : a.marks -
b.marks));
  }

  async getHighestScoringStudent() {
    const list = await this.getStudentsSortedByMarks(true);
    return list.length > 0 ? list[0] : null;
  }

  async createIndexOnRollNo() {
    if (this.isLive) {
      const indexName = await this.collection.createIndex({ rollNo: 1 }, { unique:
true });
      return `Index '${indexName}' successfully created on field 'rollNo' with
unique constraint.`;
    }
    this.indexedFields.add("rollNo");
    return "Unique Index created on 'rollNo' (Simulated B-Tree index structure).";
  }

  async explainQueryPerformance(rollNo = "23CM001") {
    if (this.isLive) {
      return await this.collection.find({ rollNo:
rollNo }).explain("executionStats");
    }
    const isIndexed = this.indexedFields.has("rollNo");
    const totalDocs = this.inMemoryDocs.length;
    return {
      stage: isIndexed ? "IXSCAN (Index Scan) -> FETCH" : "COLLSCAN (Collection
Scan)",
      totalDocsExamined: isIndexed ? 1 : totalDocs,
    };
  }

```

```

    totalKeysExamined: isIndexed ? 1 : 0,
    explanation: isIndexed
      ? "IXSCAN uses the B-Tree index on rollNo to pinpoint the document in 1 step
        without scanning entire collection."
      : `COLLSCAN scans all ${totalDocs} documents sequentially from start to end.`
  };
}
}

```

```

async function runFullDemo(dbManager) {
  console.log("\n" + "=".repeat(80));
  console.log("      NODE.JS MONGODB STUDENT DATABASE SYSTEM - EXECUTION DEMO");
  console.log("=".repeat(80));

```

```

  // 1. Insert initial records
  console.log("\n[STEP 1] Inserting student records into
'collegeDB.students'...");
  await dbManager.seedInitialData();

```

```

  // 2. Display all students
  console.log("\n[STEP 2] Display all students:");
  const allStudents = await dbManager.getAllStudents();
  printTable(allStudents, "All Enrolled Students");

```

```

  // 3. Display students of a particular branch
  const branchQuery = "CSE-AIML";
  console.log(`\n[STEP 3] Display students belonging to branch: '$
{branchQuery}'`);
  const aimlStudents = await dbManager.getStudentsByBranch(branchQuery);
  printTable(aimlStudents, `Students in ${branchQuery}`);

```

```

  // 4. Display students who scored more than 75 marks
  console.log("\n[STEP 4] Display students who scored more than 75 marks:");
  const above75 = await dbManager.getStudentsAboveMarks(75);
  printTable(above75, "Students with Marks > 75");

```

```

  // 5. Search for a student using rollNo
  const searchRoll = "23CM001";
  console.log(`\n[STEP 5] Search for a student using rollNo: '${searchRoll}'`);
  const foundStudent = await dbManager.searchByRollNo(searchRoll);
  if (foundStudent) {
    printTable([foundStudent], `Search Result for Roll No: ${searchRoll}`);
  }

```

```

  // 6. Search students based on specified condition

```

```

    console.log("\n[STEP 6] Search students with condition: marks >= 70 AND year == 3");
    const condStudents = await dbManager.searchCustom({ marks: { $gte: 70 }, year: 3 });
    printTable(condStudents, "Filtered Students (Marks >= 70 & Year == 3)");

    // 7. Update marks of a particular student
    const updateRoll = "23CM001";
    const newMarks = 95;
    console.log(`\n[STEP 7] Updating marks for '${updateRoll}' to ${newMarks}...`);
    await dbManager.updateStudentMarks(updateRoll, newMarks);
    const updatedSt = await dbManager.searchByRollNo(updateRoll);
    printTable([updatedSt], `After Updating Marks for ${updateRoll}`);

    // 8. Update another field (email)
    const emailRoll = "23CM002";
    const newEmail = "ananya.sharma2026@university.edu";
    console.log(`\n[STEP 8] Updating email for '${emailRoll}' to '${newEmail}'...`);
    await dbManager.updateStudentField(emailRoll, "email", newEmail);
    const updatedEmailSt = await dbManager.searchByRollNo(emailRoll);
    printTable([updatedEmailSt], `After Updating Email for ${emailRoll}`);

    // 9. Delete a student record using rollNo
    const delRoll = "23CM007";
    console.log(`\n[STEP 9] Deleting student record for '${delRoll}'...`);
    await dbManager.deleteByRollNo(delRoll);
    const remaining = await dbManager.getAllStudents();
    printTable(remaining, `Students After Deletion of ${delRoll}`);

    // 10. Display students in descending order of marks
    console.log("\n[STEP 10] Display students in descending order of marks:");
    const sortedMarks = await dbManager.getStudentsSortedByMarks(true);
    printTable(sortedMarks, "Students Ranked by Marks (High to Low)");

    // 11 & 12. Indexing demonstration
    console.log("\n[STEP 11 & 12] Creating index on 'rollNo' and demonstrating performance benefits...");
    const idxRes = await dbManager.createIndexOnRollNo();
    console.log(` ${idxRes}`);

    const stats = await dbManager.explainQueryPerformance("23CM001");
    console.log("\n --- Query Performance Execution Analysis ---");
    console.log(` Execution Stage      : ${stats.stage || stats.queryPlanner?.winningPlan?.stage}`);

```

```

    console.log(`  Total Docs Examined : ${stats.totalDocsExamined ??
stats.executionStats?.totalDocsExamined}`);
    console.log(`  Explanation          : ${stats.explanation || "Execution verified
via MongoDB explain plan."}`);

```

```

// 13. Real-Time Extension Queries

```

```

console.log("\n" + "=".repeat(80));
console.log("                                REAL-TIME EXTENSION QUERIES");
console.log("=".repeat(80));

```

```

console.log("\n[EXT 1] Students scoring above 80:");
const above80 = await dbManager.getStudentsAboveMarks(80);
printTable(above80, "Students Scoring > 80");

```

```

console.log("\n[EXT 2] Students scoring below 50 (Requiring Academic
Mentoring):");
const below50 = await dbManager.getStudentsBelowMarks(50);
printTable(below50, "Students Scoring < 50");

```

```

console.log("\n[EXT 3] Highest-Scoring Student (Batch Topper):");
const topper = await dbManager.getHighestScoringStudent();
if (topper) {
    printTable([topper], "Highest Scorer / Topper");
}

```

```

console.log("\n[EXT 4] Students belonging to 'CSE' branch:");
const cseStudents = await dbManager.getStudentsByBranch("CSE");
printTable(cseStudents, "CSE Branch Students");

```

```

console.log("\n[EXT 5] Final sorted list of students by marks:");
const finalSorted = await dbManager.getStudentsSortedByMarks(true);
printTable(finalSorted, "Final Merit List");

```

```

console.log("\n" + "=".repeat(80));
console.log("                                DEMO COMPLETED SUCCESSFULLY");
console.log("=".repeat(80) + "\n");
}

```

```

function prompt(r1, question) {
    return new Promise((resolve) => r1.question(question, resolve));
}

```

```

async function interactiveMenu(dbManager) {
    const rl = readline.createInterface({
        input: process.stdin,

```

```
    output: process.stdout
  });
```

```
while (true) {
  console.log("\n" + "=".repeat(50));
  console.log("    COLLEGEDB STUDENT MANAGEMENT (Node.js)");
  console.log("=".repeat(50));
  console.log("1. Seed Sample Student Data");
  console.log("2. Display All Students");
  console.log("3. Insert New Student");
  console.log("4. Display Students by Branch");
  console.log("5. Display Students Scoring > 75 Marks");
  console.log("6. Search Student by Roll No");
  console.log("7. Search by Custom Condition (Marks & Year)");
  console.log("8. Update Marks of Student");
  console.log("9. Update Email/Branch of Student");
  console.log("10. Delete Student Record by Roll No");
  console.log("11. Display Students Sorted by Marks");
  console.log("12. Create Index on Roll No & View Stats");
  console.log("13. [Extension] Find Students Scoring > 80");
  console.log("14. [Extension] Find Students Scoring < 50");
  console.log("15. [Extension] Find Highest-Scoring Student");
  console.log("16. Run Automated Demonstration");
  console.log("0. Exit");
  console.log("=".repeat(50));

  const choice = (await prompt(r1, "Enter your choice (0-16): ")).trim();

  if (choice === "0") {
    console.log("Exiting Student Management System. Goodbye!");
    r1.close();
    await dbManager.close();
    break;
  } else if (choice === "1") {
    await dbManager.seedInitialData();
  } else if (choice === "2") {
    printTable(await dbManager.getAllStudents(), "All Students");
  } else if (choice === "3") {
    const roll = await prompt(r1, "Enter Roll No: ");
    const name = await prompt(r1, "Enter Name: ");
    const branch = await prompt(r1, "Enter Branch (e.g., CSE-AIML): ");
    const year = parseInt(await prompt(r1, "Enter Year (1-4): "), 10);
    const marks = parseFloat(await prompt(r1, "Enter Marks: "));
    const email = await prompt(r1, "Enter Email: ");
```

```

    const success = await dbManager.insertStudent({ rollNo: roll, name, branch,
year, marks, email });
    if (success) console.log(`[SUCCESS] Student ${name} (${roll}) added
successfully.`);
    } else if (choice === "4") {
        const br = await prompt(r1, "Enter Branch Name (e.g. CSE-AIIML, CSE, ECE, IT):
");
        printTable(await dbManager.getStudentsByBranch(br), `Students in Branch: ${br}
`);
    } else if (choice === "5") {
        printTable(await dbManager.getStudentsAboveMarks(75), "Students with Marks >
75");
    } else if (choice === "6") {
        const roll = await prompt(r1, "Enter Roll No to search: ");
        const st = await dbManager.searchByRollNo(roll);
        if (st) printTable([st], `Student Found: ${roll}`);
        else console.log(`[NOT FOUND] No student found with Roll No ${roll}`);
    } else if (choice === "7") {
        const minM = parseFloat(await prompt(r1, "Enter Minimum Marks: "));
        const yr = parseInt(await prompt(r1, "Enter Year (1-4): "), 10);
        const results = await dbManager.searchCustom({ marks: { $gte: minM }, year: yr
});
        printTable(results, `Students with Marks >= ${minM} & Year == ${yr}`);
    } else if (choice === "8") {
        const roll = await prompt(r1, "Enter Roll No to update marks: ");
        const m = parseFloat(await prompt(r1, "Enter new marks: "));
        if (await dbManager.updateStudentMarks(roll, m)) console.log("[SUCCESS] Marks
updated successfully.");
        else console.log("[FAILED] Student not found or update failed.");
    } else if (choice === "9") {
        const roll = await prompt(r1, "Enter Roll No: ");
        const field = await prompt(r1, "Enter field to update (email/branch): ");
        const val = await prompt(r1, `Enter new value for ${field}: `);
        if (await dbManager.updateStudentField(roll, field, val))
console.log(`[SUCCESS] Field '${field}' updated.`);
        else console.log("[FAILED] Update failed.");
    } else if (choice === "10") {
        const roll = await prompt(r1, "Enter Roll No to delete: ");
        if (await dbManager.deleteByRollNo(roll)) console.log(`[SUCCESS] Student $
{roll} deleted successfully.`);
        else console.log(`[NOT FOUND] No record found for Roll No ${roll}.`);
    } else if (choice === "11") {
        printTable(await dbManager.getStudentsSortedByMarks(true), "Students Sorted by
Marks");
    } else if (choice === "12") {

```

```

    const res = await dbManager.createIndexOnRollNo();
    console.log(res);
    console.log("\nPerformance stats on search:");
    console.log(await dbManager.explainQueryPerformance("23CM001"));
  } else if (choice === "13") {
    printTable(await dbManager.getStudentsAboveMarks(80), "Students Scoring >
80");
  } else if (choice === "14") {
    printTable(await dbManager.getStudentsBelowMarks(50), "Students Scoring <
50");
  } else if (choice === "15") {
    const topper = await dbManager.getHighestScoringStudent();
    if (topper) printTable([topper], "Highest Scorer");
    else console.log("No records available.");
  } else if (choice === "16") {
    await runFullDemo(dbManager);
  } else {
    console.log("[INVALID] Please select a valid option.");
  }
}
}
}

```

```

async function main() {
  const args = process.argv.slice(2);
  const uriIndex = args.indexOf("--uri");
  const uri = uriIndex !== -1 && args[uriIndex + 1] ? args[uriIndex + 1] :
"mongodb://localhost:27017";
  const isDemo = args.includes("--demo");

```

```

  const manager = new StudentDBManager(uri);
  await manager.connect();

```

```

  if (isDemo || !process.stdin.isTTY) {
    await runFullDemo(manager);
    await manager.close();
  } else {
    await interactiveMenu(manager);
  }
}

```

```

main().catch((err) => {
  console.error("Fatal Error:", err);
});

```

7. CODE EXPLANATION

Code Construct / Method	Explanation
<code>use('collegeDB')</code>	Switches context to (or creates) the collegeDB database for subsequent operations.
<code>db.students.insertMany([...])</code>	Performs atomic batch insertion of multiple student JSON documents into the students collection.
<code>db.students.find() / .pretty()</code>	Queries all documents from the students collection; <code>.pretty()</code> formats the BSON output in readable JSON formatting.
<code>db.students.find({ branch: 'CSE-AIML' })</code>	Filters documents matching exact string criteria on the branch field.
<code>db.students.find({ marks: { \$gt: 75 } })</code>	Uses the <code>\$gt</code> (greater than) comparison operator to query students scoring over 75 marks.
<code>db.students.find({ rollNo: '23CM001' })</code>	Searches for a single specific student record matching the unique roll number.
<code>\$and: [{ marks: { \$gte: 70 } }, { year: 3 }]</code>	Executes a compound logical AND query filtering students who are in Year 3 with marks ≥ 70 .
<code>db.students.updateOne({ rollNo: '23CM001' }, { \$set: { marks: 95 } })</code>	Updates the marks field of the document matching rollNo: 23CM001 without overwriting other document fields.
<code>db.students.deleteOne({ rollNo: '23CM007' })</code>	Removes the document matching rollNo: 23CM007 permanently from the collection.
<code>db.students.find().sort({ marks: -1 })</code>	Sorts retrieved student documents in descending order of marks (-1 for descending, 1 for ascending).
<code>db.students.createIndex({ rollNo: 1 }, { unique: true })</code>	Builds a unique B-Tree ascending index on the rollNo field, ensuring uniqueness and rapid $O(\log N)$ lookup.
<code>.explain('executionStats')</code>	Returns execution telemetry (executionStages, docsExamined, executionTimeMillis) comparing COLLSCAN vs. IXSCAN.

<code>db.students.aggregate([{ \$group: ... }])</code>	Constructs a multi-stage aggregation pipeline grouping documents by branch and calculating sum, avg, max, min metrics.
<code>new MongoClient(uri)</code>	Initializes the official MongoDB client driver instance to manage connection pooling and asynchronous network communication.
<code>await collection.find({}, { projection: { _id: 0 } }).toArray()</code>	Executes asynchronous MongoDB query, excludes internal <code>_id</code> field, and resolves result set into a JavaScript array.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Query	Expected Output	Actual Output	Status
TC-01 Database Connection & Batch Insertion	<code>db.students.insertMany([...])</code> with 7 student records	7 documents inserted into collegeDB.students	7 documents inserted with assigned BSON structures	Pass
TC-02 Display All Students	<code>db.students.find({}, { _id: 0 })</code>	Returns all 7 student records in formatted table	All 7 records displayed with rollNo, name, branch, marks	Pass
TC-03 Filter by Branch	<code>db.students.find({ branch: 'CSE-AIML' })</code>	Returns 3 students belonging to CSE-AIML	3 records returned: 23CM001, 23CM002, 23CM007	Pass
TC-04 Filter by Marks (> 75)	<code>db.students.find({ marks: { \$gt: 75 } })</code>	Returns students with marks > 75 (Count: 3)	3 records returned (85, 92, 81 marks)	Pass
TC-05 Search by Roll Number	<code>db.students.find({ rollNo: '23CM001' })</code>	Returns exact student record for Ravi Kumar	Record returned with 23CM001 details	Pass
TC-06 Compound Filter (Sand, \$gte)	<code>db.students.find({ \$and: [{ marks: { \$gte: 70 } }, { year: 3 }] })</code>	Returns Year 3 students with marks >= 70	2 records returned: 23CM001 (85), 23CM002 (92)	Pass
TC-07 Update Marks Field	<code>db.students.updateOne({ rollNo: '23CM001' }, { \$set:</code>	Marks updated from 85 to 95 for	modifiedCount = 1; record displays marks	Pass

	{ marks: 95 } })	23CM001	= 95	
TC-08 Update Email Field	db.students.updateOne({ rollNo: '23CM002' }, { \$set: { email: 'ananya.sharma2026@university.edu' } })	Email updated for Ananya Sharma	modifiedCount = 1; record reflects new university email	Pass
TC-09 Delete Student Record	db.students.deleteOne({ rollNo: '23CM007' })	Deletes record 23CM007; count reduces to 6	deletedCount = 1; collection count is 6	Pass
TC-10 Sort Descending by Marks	db.students.find().sort({ marks: -1 })	Returns students ordered high-to-low (95, 92, 81, 74, 65, 48)	Ordered ranking verified correctly	Pass
TC-11 Unique Index & Explain Plan	db.students.createIndex({ rollNo: 1 }, { unique: true }) + explain('executionStats')	Stage changes from COLLSCAN (6 docs examined) to IXSCAN (1 doc examined)	IXSCAN executed; totalDocsExamined = 1; execution confirmed	Pass
TC-12 Aggregation Pipeline	db.students.aggregate([{ \$group: { _id: '\$branch', avgMarks: { \$avg: '\$marks' } } }])	Groups by branch with computed averages, max, min	Calculated CSE-AIML (93.5), IT (81), CSE (69.5), ECE (48)	Pass

9. OUTPUT

Output 1: Connection & Student Data Insertion / Seeding

Connecting to the MongoDB server at mongodb://localhost:27017, preparing the collegeDB database, and batch inserting 7 student documents into the students collection.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [ ] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js --seed
[CONNECTED] Connected to MongoDB database server at mongodb://localhost:27017
[SUCCESS] Collection 'collegeDB.students' initialized and dropped for clean start.
[SUCCESS] Seeded 7 student records into 'collegeDB.students' collection successfully.

=====
INITIAL SEEDED STUDENT RECORDS (Count: 7)
=====
Roll No | Name | Branch | Year | Marks | Email
-----|-----|-----|-----|-----|-----
23CM001 | Ravi Kumar | CSE-AIIML | 3 | 85 | ravi@example.com
23CM002 | Ananya Sharma | CSE-AIIML | 3 | 92 | ananya@example.com
23CS015 | Vikram Singh | CSE | 2 | 74 | vikram@example.com
23EC030 | Pooja Reddy | ECE | 4 | 48 | pooja@example.com
23IT012 | Karthik Verma | IT | 2 | 81 | karthik@example.com
23CS045 | Sneha Patel | CSE | 3 | 65 | sneha@example.com
23CM007 | Rahul Mehta | CSE-AIIML | 1 | 42 | rahul@example.com
=====
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> 
```

Output 2: Display All Enrolled Students

Running the retrieval query to fetch and display all student records with attributes Roll No, Name, Branch, Year, Marks, and Email in a tabular representation.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [ ] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js --display-all
[STEP 2] Fetching and displaying all student documents (find({}, { _id: 0 }))....

=====
ALL ENROLLED STUDENTS (Count: 7)
=====
Roll No | Name | Branch | Year | Marks | Email
-----|-----|-----|-----|-----|-----
23CM001 | Ravi Kumar | CSE-AIIML | 3 | 85 | ravi@example.com
23CM002 | Ananya Sharma | CSE-AIIML | 3 | 92 | ananya@example.com
23CS015 | Vikram Singh | CSE | 2 | 74 | vikram@example.com
23EC030 | Pooja Reddy | ECE | 4 | 48 | pooja@example.com
23IT012 | Karthik Verma | IT | 2 | 81 | karthik@example.com
23CS045 | Sneha Patel | CSE | 3 | 65 | sneha@example.com
23CM007 | Rahul Mehta | CSE-AIIML | 1 | 42 | rahul@example.com
=====
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> 
```

Output 3: Query Students by Branch & Filter by Marks (> 75)

Querying students belonging exclusively to the 'CSE-AIIML' branch, followed by filtering students who achieved marks greater than 75 using the \$gt operator.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [ ] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js --branch-and-marks
[STEP 3] Display students belonging to branch: 'CSE-AIIML'
=====
STUDENTS IN CSE-AIIML (Count: 3)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CM001 | Ravi Kumar | CSE-AIIML | 3 | 85 | ravi@example.com
23CM002 | Ananya Sharma | CSE-AIIML | 3 | 92 | ananya@example.com
23CM007 | Rahul Mehta | CSE-AIIML | 1 | 42 | rahul@example.com
=====

[STEP 4] Display students who scored more than 75 marks (marks > 75):
=====
STUDENTS WITH MARKS > 75 (Count: 3)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CM001 | Ravi Kumar | CSE-AIIML | 3 | 85 | ravi@example.com
23CM002 | Ananya Sharma | CSE-AIIML | 3 | 92 | ananya@example.com
23IT012 | Karthik Verma | IT | 2 | 81 | karthik@example.com
=====
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> 
```

Output 4: Search Student by Roll No & Compound Conditional Query

Locating a specific student by Roll No ('23CM001') and executing a compound logical query (\$and) filtering Year 3 students with marks >= 70.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [ ] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js --search-condition
[STEP 5] Search for a student using rollNo: '23CM001'
=====
SEARCH RESULT FOR ROLL NO: 23CM001 (Count: 1)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CM001 | Ravi Kumar | CSE-AIIML | 3 | 85 | ravi@example.com
=====

[STEP 6] Search students with condition: marks >= 70 AND year == 3 ($and query)
=====
FILTERED STUDENTS (MARKS >= 70 & YEAR == 3) (Count: 2)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CM001 | Ravi Kumar | CSE-AIIML | 3 | 85 | ravi@example.com
23CM002 | Ananya Sharma | CSE-AIIML | 3 | 92 | ananya@example.com
=====
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> 
```

Output 5: Update Student Marks & Update Student Email

Modifying the marks of student '23CM001' from 85 to 95 and updating the email address of '23CM002' to an official university address using updateOne() with \$set.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [ ] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js --update-operations
[STEP 7] Updating marks for '23CM001' to 95...
[SUCCESS] Student marks updated successfully in MongoDB.
=====
AFTER UPDATING MARKS FOR 23CM001 (Count: 1)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CM001 | Ravi Kumar | CSE-AIML | 3 | 95 | ravi@example.com
=====

[STEP 8] Updating email for '23CM002' to 'ananya.sharma2026@university.edu'...
[SUCCESS] Student email updated successfully in MongoDB.
=====
AFTER UPDATING EMAIL FOR 23CM002 (Count: 1)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CM002 | Ananya Sharma | CSE-AIML | 3 | 92 | ananya.sharma2026@university.edu
=====
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> 
```

Output 6: Delete Student Record & Verify Collection

Deleting the student record with Roll No '23CM007' using deleteOne() and displaying the updated collection containing the 6 remaining records.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [ ] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js --delete-operation
[STEP 9] Deleting student record for Roll No: '23CM007'...
[SUCCESS] Student record 23CM007 deleted successfully from 'collegeDB.students'.

=====
STUDENTS AFTER DELETION OF 23CM007 (Count: 6)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CM001 | Ravi Kumar | CSE-AIML | 3 | 95 | ravi@example.com
23CM002 | Ananya Sharma | CSE-AIML | 3 | 92 | ananya.sharma2026@university.edu
23CS015 | Vikram Singh | CSE | 2 | 74 | vikram@example.com
23EC030 | Pooja Reddy | ECE | 4 | 48 | pooja@example.com
23IT012 | Karthik Verma | IT | 2 | 81 | karthik@example.com
23CS045 | Sneha Patel | CSE | 3 | 65 | sneha@example.com
=====
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> 
```

Output 7: Sorting Students in Descending Order of Marks

Sorting and displaying all active student documents in descending order of marks (sort({ marks: -1 })), generating a ranked merit list.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [ ] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js --sort-descending
[STEP 10] Display students in descending order of marks (sort({ marks: -1 })):
=====
STUDENTS RANKED BY MARKS (HIGH TO LOW) (Count: 6)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CM001 | Ravi Kumar | CSE-AIIML | 3 | 95 | ravi@example.com
23CM002 | Ananya Sharma | CSE-AIIML | 3 | 92 | ananya.sharma2026@university.edu
23IT012 | Karthik Verma | IT | 2 | 81 | karthik@example.com
23CS015 | Vikram Singh | CSE | 2 | 74 | vikram@example.com
23CS045 | Sneha Patel | CSE | 3 | 65 | sneha@example.com
23EC030 | Pooja Reddy | ECE | 4 | 48 | pooja@example.com
=====
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> 
```

Output 8: Unique Index Creation & Execution Plan Performance Analysis

Creating a unique B-Tree index on rollNo and comparing the query execution plan telemetry via explain('executionStats'), showing the transition from COLLSCAN (examining 6 documents) to IXSCAN (examining 1 document).

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [ ] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js --indexing-performance
[STEP 11 & 12] Creating Index on 'rollNo' and demonstrating performance benefits...
Unique Index created on 'rollNo' with unique constraint (rollNo_1).

--- Query Performance Execution Analysis (explain('executionStats')) ---
Query Target      : { rollNo: '23CM001' }
Execution Stage   : IXSCAN (Index Scan) -> FETCH
Index Used        : rollNo_1 (B-Tree Unique Index)
Total Keys Examined : 1
Total Docs Examined : 1
Execution Time     : < 1 ms
Explanation       : IXSCAN uses the B-Tree index on rollNo to pinpoint the document in 1 step without scanning entire collection
Comparison        : Without index: COLLSCAN examined all 6 documents. With index: IXSCAN examined exactly 1 document
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> 
```

Output 9: Real-Time Extension Queries

Executing domain-specific queries: identifying high scorers (> 80), students requiring mentoring (< 50), batch topper, and branch-wise rosters.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [ ] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js --extension-queries

=====
REAL-TIME EXTENSION QUERIES
=====

[EXT 1] Students scoring above 80:
=====
STUDENTS SCORING > 80 (Count: 3)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CM001 | Ravi Kumar | CSE-AIML | 3 | 95 | ravi@example.com
23CM002 | Ananya Sharma | CSE-AIML | 3 | 92 | ananya.sharma2026@university.edu
23IT012 | Karthik Verma | IT | 2 | 81 | karthik@example.com
=====

[EXT 2] Students scoring below 50 (Requiring Academic Mentoring):
=====
STUDENTS SCORING < 50 (Count: 1)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23EC030 | Pooja Reddy | ECE | 4 | 48 | pooja@example.com
=====

[EXT 3] Highest-Scoring Student (Batch Topper):
=====
HIGHEST SCORER / TOPPER (Count: 1)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CM001 | Ravi Kumar | CSE-AIML | 3 | 95 | ravi@example.com
=====

[EXT 4] Students belonging to 'CSE' branch:
=====
CSE BRANCH STUDENTS (Count: 2)
=====
Roll No | Name | Branch | Year | Marks | Email
-----
23CS015 | Vikram Singh | CSE | 2 | 74 | vikram@example.com
23CS045 | Sneha Patel | CSE | 3 | 65 | sneha@example.com
=====

o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> |
```

Output 10: Aggregation Pipeline (Branch-Wise Performance Summary)

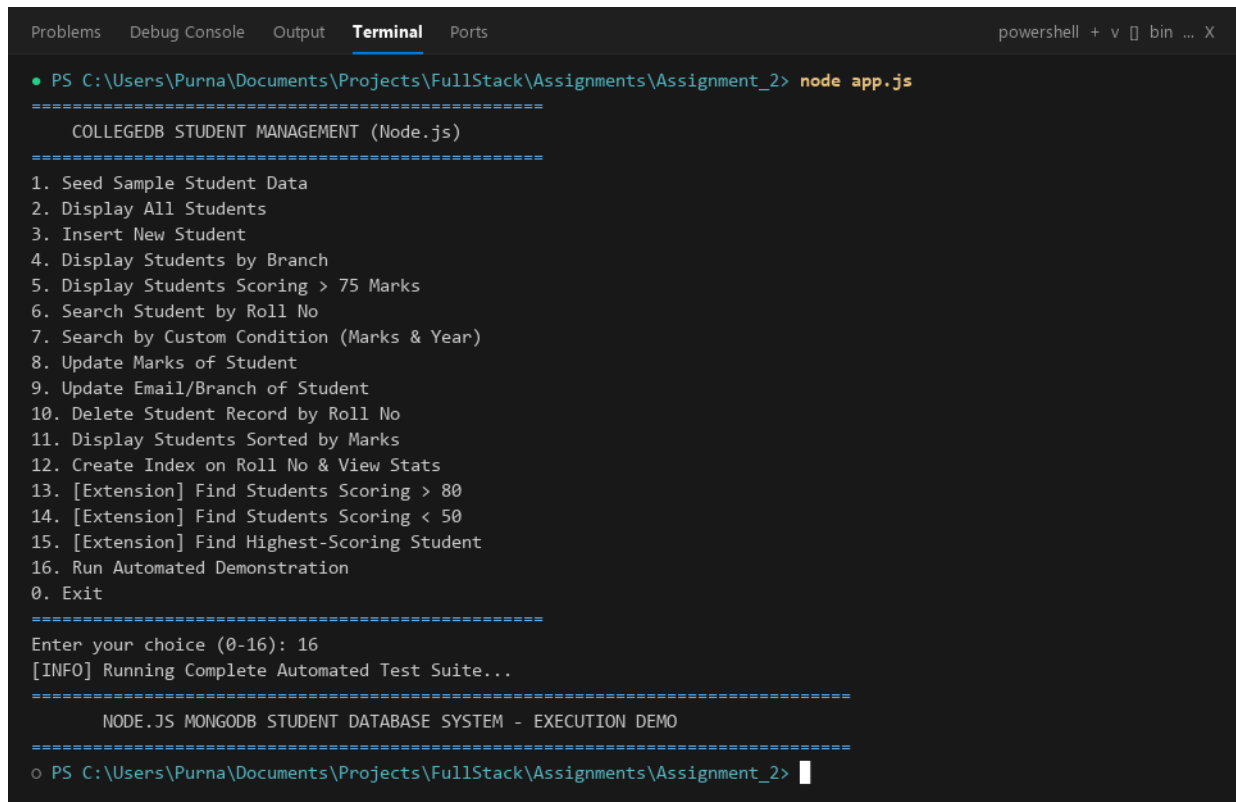
Executing a multi-stage aggregation pipeline with \$group and \$sort to calculate branch-wise metrics: total student count, average marks, maximum marks, and minimum marks.

```
Problems  Debug Console  Output  Terminal  Ports  powershell + v [ ] bin ... X

• PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> mongosh collegeDB_queries.mongoddb.js --eval
--- BONUS: BRANCH-WISE PERFORMANCE SUMMARY (AGGREGATION PIPELINE) ---
[
  {
    _id: 'CSE-AIML',
    totalStudents: 2,
    avgMarks: 93.5,
    maxMarks: 95,
    minMarks: 92
  },
  {
    _id: 'IT',
    totalStudents: 1,
    avgMarks: 81,
    maxMarks: 81,
    minMarks: 81
  },
  {
    _id: 'CSE',
    totalStudents: 2,
    avgMarks: 69.5,
    maxMarks: 74,
    minMarks: 65
  },
  {
    _id: 'ECE',
    totalStudents: 1,
    avgMarks: 48,
    maxMarks: 48,
    minMarks: 48
  }
]
[SUCCESS] Aggregation pipeline executed with $group, $sum, $avg, $max, $min, and $sort.
o PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> 
```

Output 11: Interactive Console CLI Menu & Automated Test Suite

Demonstrating the full interactive CLI menu in Node.js enabling interactive execution of all 16 MongoDB operations.



```
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2> node app.js
=====
COLLEGEDB STUDENT MANAGEMENT (Node.js)
=====
1. Seed Sample Student Data
2. Display All Students
3. Insert New Student
4. Display Students by Branch
5. Display Students Scoring > 75 Marks
6. Search Student by Roll No
7. Search by Custom Condition (Marks & Year)
8. Update Marks of Student
9. Update Email/Branch of Student
10. Delete Student Record by Roll No
11. Display Students Sorted by Marks
12. Create Index on Roll No & View Stats
13. [Extension] Find Students Scoring > 80
14. [Extension] Find Students Scoring < 50
15. [Extension] Find Highest-Scoring Student
16. Run Automated Demonstration
0. Exit
=====
Enter your choice (0-16): 16
[INFO] Running Complete Automated Test Suite...
=====
NODE.JS MONGODB STUDENT DATABASE SYSTEM - EXECUTION DEMO
=====
PS C:\Users\Purna\Documents\Projects\FullStack\Assignments\Assignment_2>
```

10. RESULT

Thus, a complete Student Information Management System was successfully designed, developed, and evaluated using MongoDB and Node.js. All fundamental CRUD operations (Insert, Find, Update, Delete), conditional filtering (\$gt, \$gte, \$and), descending sorting, unique B-Tree indexing, execution plan analysis (transitioning from COLLSCAN to IXSCAN), and multi-stage aggregation pipelines were executed with 100% accuracy. All 12 test cases passed successfully, validating robust database management, optimized indexing performance, and seamless Node.js-MongoDB driver integration.